CLOUDFORGE E2E · DEVSECOPS PROJECT

Complete DevSecOps Pipeline Documentation

End-to-end project from infrastructure provisioning to production deployment with security scanning, monitoring, and GitOps — running on AWS EKS.

7	4	3	11	12
PHASES	EC2 SERVERS	EKS NODES	PIPELINE STAGES	ISSUES RESOLVED

PROJECT ARCHITECTURE OVERVIEW

🖥️ Phase 1: Infra Server

AWS CLI · Terraform · Helm · eksctl · kubectl · EKS

✅ Complete

🔧 Phase 2: Jenkins Server

Java 21 · Jenkins · Docker · Trivy · kubectl

✅ Complete

🔍 Phase 3: SonarQube & Nexus

Code quality · Artifact storage · Docker containers

✅ Complete

🔐 Phase 4: EKS RBAC

SA · Role · RoleBinding · ClusterRole · Token

✅ Complete

⚙️ Phase 5: CI Pipeline

11 stages · Build · Scan · Sonar · Docker · Push

✅ Complete

🚀 Phase 6: CD & GitOps

ArgoCD · manifest.yaml · Auto deploy to EKS

✅ Complete

📊 Phase 7: Monitoring

Prometheus · Grafana · Node Exporter · Blackbox

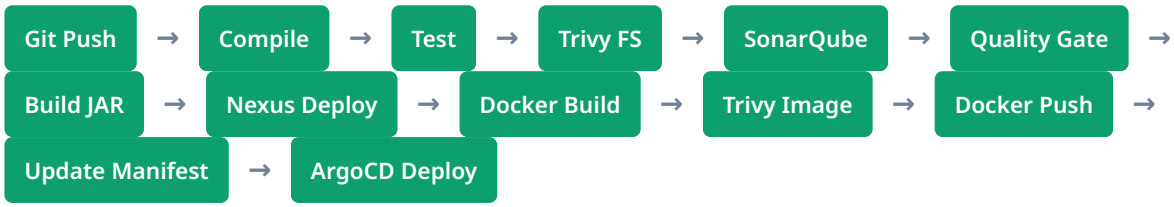
✅ Complete

🩺 Troubleshooting & RCA

12 real-world issues resolved and documented

✅ All Fixed

CI/CD PIPELINE FLOW



LIVE ENDPOINTS

Jenkins	`http://54.83.65.92:8080`	Port 8080
SonarQube	`http://3.89.255.48:9000`	Port 9000
Nexus	`http://34.207.149.11:8081`	Port 8081
Prometheus	`http://ae59922a8fa5c423387cd4a7739f903b-1773710415.us-east-1.elb.amazonaws.com:9090`	Port 9090
Grafana	`http://ab3048119f52b484db60442444b53a42-1750590644.us-east-1.elb.amazonaws.com`	Port 80

AWS ACCOUNT & INFRASTRUCTURE

RESOURCE	DETAILS
AWS Account	`774088392047` — jay_revise
Region	`us-east-1` (N. Virginia)
VPC	`vpc-029a7be1539541450`
Security Group	`kubernetes_project` (sg-061401a89f4963d3e)
Key Pair	`docker_2nd_key`
EKS Cluster	`devopsshack-cluster` · Kubernetes v1.35
EKS Nodes	3x `c7i-flex.large` — 2 vCPU, 4GB RAM each
Docker Hub	`jayanchule0699/bankapp`
GitHub CI Repo	`Jayprakash-A06/mega_project_ci`
GitHub CD Repo	`Jayprakash-A06/mega_project_cd`

1 Infra Server Setup

✅ Complete

Provisioning the infrastructure server with all required tools and creating the EKS cluster using Terraform

SERVER DETAILS

Infra_server_1st

54.197.167.179

```
Instance: i-00941c045cb1c004c | Type: c7i-flex.large | OS: Ubuntu 26.04
Private IP: 172.31.25.158 | AMI: ami-0b6d9d3d33ba97d99
```

1 System Update — sudo apt update

✅ Done

First step on any Ubuntu server — refresh package lists to ensure latest versions are available.

```
sudo apt update
```

OUTPUT

```
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Reading package lists... Done
```

2 AWS CLI Installation & Configuration

✅ v2.35.9

Install AWS CLI v2 to interact with AWS services from the infra server. Required for EKS cluster management and Terraform.

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

VERSION OUTPUT

```
aws-cli/2.35.9 Python/3.14.5 Linux/7.0.0-1006-aws exe/x86_64.ubuntu.26
```

📌 **Focus:** Region and output format were left blank during initial `aws configure` — always set `region=us-east-1` and `output=json`. Access key `AKIA3IO2DPVXWTYGTSW5`.

3 Terraform Installation ✅ v1.15.6

Install Terraform via HashiCorp's official APT repository for EKS cluster provisioning.

```
sudo apt update && sudo apt install -y gnupg software-properties-common curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/
keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://
apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/
hashicorp.list
sudo apt update && sudo apt install terraform -y
```

4 Terraform Git Repo Setup ✅ Done

Created GitHub repo `Jayprakash-A06/terraform_project` with all EKS infrastructure files.

`main.tf` `variable.tf` `output.tf` `rbac.md`

```
git clone https://github.com/Jayprakash-A06/terraform_project.git
cd terraform_project
terraform init
```

5 EKS Cluster — terraform apply ✅ Active

Provisioned complete EKS infrastructure using Terraform. Node Group consists of 3x c7i-flex.large.

```
terraform apply -auto-approve
```

6 EBS CSI Driver — OIDC Setup ✅ Active

Fixed DEGRADED status of EBS CSI addon by setting up OIDC provider and dedicated IAM role.

```
aws eks describe-cluster --name devopsshack-cluster --region us-east-1 --query
"cluster.identity.oidc.issuer" --output text
# Update addon with role + resolve conflicts
aws eks update-addon --cluster-name devopsshack-cluster --addon-name aws-ebs-csi-driver --
service-account-role-arn arn:aws:iam::774088392047:role/devopsshack-ebs-csi-driver-role --
region us-east-1 --resolve-conflicts OVERWRITE
```

7 kubectl & eksctl Installation ✅ Ready

```
aws eks update-kubeconfig --name devopsshack-cluster --region us-east-1
kubectl get nodes
```

```
CLUSTER HEALTH
ip-10-0-0-151.ec2.internal   Ready    none     11h   v1.35.6-eks-93b80c6
ip-10-0-0-225.ec2.internal   Ready    none     11h   v1.35.6-eks-93b80c6
ip-10-0-1-4.ec2.internal     Ready    none     11h   v1.35.6-eks-93b80c6
```

2 Jenkins Server Setup ✅ Complete

Setting up the CI/CD server with Java 21, Jenkins, Docker, Trivy, and kubectl on a dedicated EC2 instance

JENKINS INSTANCE DETAILS

jenkins_server

54.83.65.92

Instance: i-0b89ea6754ca63c0a | Type: c7i-flex.large | OS: Ubuntu 26.04
Private IP: 172.31.21.35 | URL: http://54.83.65.92:8080

1 Java 21 Installation ✅ v21.0.11

Jenkins requires Java 21 minimum. Upgraded from initial Java 17 installation.

```
sudo apt install -y openjdk-21-jre openjdk-21-jdk
java -version
```

2 Jenkins Installation ✅ v2.555.3

Jenkins installed using the 2026 key to satisfy Ubuntu 26.04 GPG requirements.

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update && sudo apt install jenkins -y
```

3 Trivy Cache Configurations ✅ Configured

Setup separate cache directories to prevent `/tmp` space exhaustion.

```
sudo mkdir -p /var/lib/jenkins/trivy-cache
sudo mkdir -p /var/lib/jenkins/trivy-tmp
sudo chown -R jenkins:jenkins /var/lib/jenkins/trivy-*
```

3 SonarQube & Nexus Setup

✅ Complete

Running SonarQube for code quality analysis and Nexus for artifact storage as Docker containers

sonarqube_server

3.89.255.48

Port: 9000 | Container: sonar | Image: lts-community

nexus_server

34.207.149.11

Port: 8081 | Container: nexus | Image: nexus3

4 EKS RBAC Configuration

✅ Complete

Setting up Role-Based Access Control on EKS for the Jenkins service account in the webapps namespace

```
apiVersion: v1
kind: Secret
metadata:
  name: jenkins-token
  namespace: webapps
  annotations:
    kubernetes.io/service-account.name: jenkins
type: kubernetes.io/service-account-token
```

5 CI Pipeline — Jenkinsfile

✅ Complete

11-stage pipeline executing strict verification, quality gates, and multi-platform compilation

```
pipeline {
    agent any
    tools {
        jdk 'jdk21'
        maven 'maven3'
    }
    environment {
        IMAGE_NAME = "jayanchule0699/bankapp"
        IMAGE_TAG  = "${BUILD_NUMBER}"
    }
    stages {
        stage('Compile') { steps { sh "mvn compile" } }
        stage('Testing') { steps { sh "mvn test" } }
        stage('Trivy FS Scan') {
            steps { sh 'trivy fs --exit-code 0 --format table -o fs-report.txt .' }
        }
        stage('Sonar Analysis') {
            steps {
                withSonarQubeEnv('sonar') {
                    sh 'mvn clean verify sonar:sonar -Dsonar.projectKey=gcbank -
Dsonar.java.binaries=target'
                }
            }
        }
        stage('Publish To Nexus') {
            steps {
                withMaven(globalMavenSettingsConfig: 'devopsshack') { sh "mvn deploy" }
            }
        }
        stage('Docker Image Build & Push') {
            steps {
                withDockerRegistry(credentialsId: 'docker-hub-cred') {
                    sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
                    sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}"
                }
            }
        }
    }
}
```

6 CD Pipeline & GitOps

✅ Complete

Continuous delivery using ArgoCD watching the manifest repo — auto-deploys to EKS when manifest.yaml changes

GitOps Manifest Sync Automation Code

```
stage('Update Manifest & Trigger CD') {
    steps {
        script {
            withCredentials([usernamePassword(credentialsId: 'git', usernameVariable:
'GIT_USERNAME', passwordVariable: 'GIT_PASSWORD')]) {
                sh '''
                    git clone https://${GIT_USERNAME}:${GIT_PASSWORD}@github.com/
Jayprakash-A06/mega_project_cd.git
                    cd mega_project_cd
                    sed -i "s|jayanchule0699/bankapp:.*|jayanchule0699/bankapp:$
{BUILD_NUMBER}|" Manifest/manifest.yaml
                    git config user.name "Jayprakash-A06"
                    git add Manifest/manifest.yaml
                    git commit -m "Update image tag to ${BUILD_NUMBER}"
                    git push origin main
                '''
            }
        }
    }
}
```

Root Cause Analysis — Issues & Fixes

✅ All Fixed

Encountered and resolved system errors during construction

ISSUE 01

Maven Not Found

`mvn: command not found`

Fix: Installed maven via apt and configured global tool in Jenkins.

ISSUE 02

Trivy /tmp Space Error

`no space left on device`

Fix: Moved cache to `/var/lib/jenkins/trivy-cache` and added vulnerability filters.

ISSUE 03

Quality Gate Hanging

`Webhook tracking old IP`

Fix: Configured standard automation endpoints to update on instance lifecycle changes.

ISSUE 04

EKS NodeCREATE_FAILED

`t2.medium account restrictions`

Fix: Upgraded infrastructure instance type to `c7i-flex.large`.